

# PROGRAMMATION ORIENTÉE OBJET EN C#

Ce document de référence synthétise les principes fondamentaux, les structures de données et les mécanismes de programmation orientée objet (POO) au sein de l'écosystème C#, tels qu'exposés dans le cadre de la formation CogniTIC.

## RÉSUMÉ EXÉCUTIF

La programmation orientée objet en C# repose sur une structure logique rigoureuse visant la modularité, la protection des données et la réutilisabilité du code. Les points clés de cette synthèse incluent :

- **Organisation Logique** : L'utilisation des espaces de noms (namespaces) pour structurer le code de manière similaire à un système de fichiers, mais sur un plan purement logique.
- **Protection et Intégrité** : Le principe d'encapsulation, soutenu par six niveaux d'accessibilité, garantit le contrôle strict des données via les propriétés et leurs accesseurs.
- **Hierarchie et Évolution** : L'héritage simple, le polymorphisme et l'utilisation de classes abstraites permettent de définir des modèles communs tout en autorisant des spécialisations précises.
- **Contrats et Abstraction** : Les interfaces agissent comme des contrats de service, facilitant l'abstraction de code et offrant une solution au manque d'héritage multiple.

## I. ORGANISATION ET STRUCTURE DU CODE

### LES ESPACES DE NOMS (NAMESPACES)

Les espaces de noms structurent le Framework .Net. Ils fonctionnent comme des conteneurs logiques (comparables à des répertoires) pouvant inclure d'autres espaces de noms ou des types de données (classes, interfaces, structures, etc.).

- **Mot-clé namespace** : Détermine l'appartenance d'un type. Le nom complet d'un type est alors Namespace.NomDuType.
- **Mot-clé using** : Permet de simplifier l'écriture en important les types d'un espace de noms, évitant ainsi de devoir saisir le nom complet à chaque appel.
- **Règles de fonctionnement** :
  - **Portée** : Descendante (les enfants connaissent les types des parents sans using).
  - **Dissimulation** : Un type dans un sous-espace de noms masque un type homonyme situé plus haut dans l'arborescence.
  - **Répétition** : Possible tant qu'il n'y a pas de conflit de types.
  - **Imbrication** : Un espace de noms peut être déclaré dans un autre.
  - **Alias** : Utilisés via using (ex: using E = Microsoft.Excel;) pour résoudre les ambiguïtés entre types homonymes provenant de sources différentes.

## NOTIONS DE CLASSE ET INSTANCIATION

Une classe est un modèle pour créer des objets de type référence. Elle définit les données (champs, propriétés) et les fonctionnalités (méthodes).

- **Classe Partielle ( partial )** : Permet de diviser la déclaration d'une classe sur plusieurs fichiers. Le compilateur les rassemble lors de la génération.
- **Instanciación** : La création d'une instance en mémoire se fait impérativement via le mot-clé new.

## 2. ENCAPSULATION ET CONTRÔLE D'ACCÈS

### LE PRINCIPE D'ENCAPSULATION

L'encapsulation protège l'intégrité des informations en limitant l'accès aux membres d'un type via des modificateurs d'accès. | Niveau d'Accessibilité | Signification || ----- | ----- || public | Accès non limité. || protected | Restreint à la classe conteneur ou aux types dérivés (héritage). || internal | Restreint à l'assembly courant (DLL ou EXE). || protected internal | Restreint à l'assembly courant ou aux types dérivés. || private | Restreint au type conteneur uniquement (classe ou structure). || private protected | Restreint aux types dérivés au sein du même assembly (C# 7.2+). |

- **Niveaux par défaut** : En l'absence de spécification, le niveau est internal pour les types de niveau supérieur et private pour les membres de classe.

### LES PROPRIÉTÉS ET INDEXEURS

Les propriétés assurent le contrôle des variables de classe via deux accesseurs :

- **get (Accesseur)** : Retourne la valeur.
- **set (Mutateur)** : Valide et assigne la valeur.
- **Auto-propriétés** : Version simplifiée (public string Nom { get; set; }) pour les cas sans logique de test spécifique. **Les Indexeurs** permettent d'indexer les instances d'une classe comme des tableaux en utilisant le mot-clé this type param. Ils peuvent être surchargés pour accepter différents types d'index.

## 3. MÉCANISMES D'HÉRITAGE ET DE FLEXIBILITÉ

### L'HÉRITAGE SIMPLE

C# ne supporte que l'héritage simple (une seule classe parente). La classe dérivée gagne tous les membres de la classe de base (sauf constructeurs et destructeurs).

- **System.Object** : Classe de base universelle dont tout élément C# hérite par défaut.
- **Redéfinition vs Dissimulation** :
- **Redéfinition ( virtual / override )** : Modifie le comportement d'une méthode de base signalée comme redéfinissable.
- **Dissimulation ( new )** : Masque explicitement un membre hérité. À utiliser avec prudence car elle peut entraîner l'exécution de la méthode de base au lieu de la méthode dérivée selon le type de la variable.
- **base** : Mot-clé pour accéder aux membres de la classe parente.
- **sealed** : Empêche l'héritage d'une classe ou la substitution ultérieure d'une méthode.

## LE POLYMORPHISME

Par héritage, un objet peut être utilisé sous plusieurs types (son propre type, ses types parents ou ses interfaces).

- **Casting Implicite** : Conversion automatique vers un type de base (ex: un Carre traité comme un object).
- **Casting Explicite** : Nécessaire pour récupérer les membres spécifiques du type d'origine après un casting implicite.

## 4. TYPES SPÉCIALISÉS ET ABSTRACTION

### CLASSES ABSTRAITES ET STATIQUES

- **Classes Abstraites ( abstract )** : Destinées uniquement à l'héritage, elles ne peuvent pas être instanciées. Elles peuvent contenir des membres abstraits (sans corps) qui *doivent* être implémentés par la première classe dérivée non abstraite.
- **Classes Statiques ( static )** : Ne peuvent pas être instanciées et ne contiennent que des membres statiques. Elles sont sealed et servent souvent de conteneurs pour des méthodes utilitaires (ex: classe Math). Les membres statiques appartiennent au type lui-même et non à une instance.

### LES INTERFACES

L'interface définit un groupe de fonctionnalités (méthodes, propriétés, événements, indexeurs) sans les implémenter. Elle agit comme un **contrat**.

- **Règles** : Pas de variables membres (champs), tous les membres sont public par défaut, et aucun bloc d'instructions n'est autorisé.
- **Avantages** : Permet l'implémentation multiple (une classe peut implémenter plusieurs interfaces), facilite l'abstraction de code et sert de filtre d'accès aux membres.

### SURCHARGE D'OPÉRATEURS

C# permet de surcharger certains opérateurs (unitaires comme ++, binaires comme +, ==) pour donner aux types personnalisés un comportement naturel.

- La surcharge se fait via des méthodes static.
- Certains opérateurs doivent être surchargés par paires (ex: == et !=).